SAFETY CHECKING IN OPEN MULTI-AGENT SYSTEMS (OMAS)

CS4099 Project Final Report

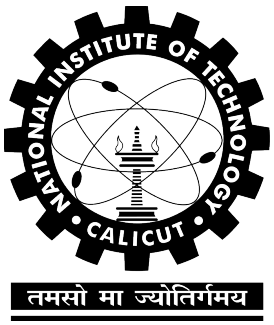
Submitted by

Sandesh Kumar **Reg No: B220510CS**

Sandeep Kharwar **Reg No: B220509CS**

Abhishek Kumar **Reg No: B220628CS**

Under the Guidance of
Dr. S Sheerazuddin



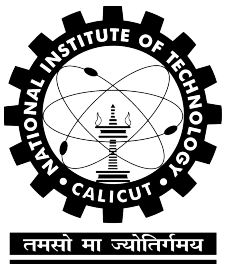
Department of Computer Science and Engineering
National Institute of Technology Calicut
Calicut, Kerala, India – 673 601

May 13, 2026

NATIONAL INSTITUTE OF TECHNOLOGY CALICUT

Kerala, India – 673 601

Department of Computer Science and Engineering



May 13, 2026

CERTIFICATE

Certified that this is a bonafide record of the project work titled

SAFETY CHECKING IN OPEN MULTI-AGENT SYSTEMS (OMAS)

done by

Sandesh Kumar
Sandeep Kharwar
Abhishek Kumar

of eighth semester B. Tech in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of the National Institute of Technology Calicut

Project Guide	**Head of Department**
Dr. S Sheerazuddin	Dr. Subashini R
Associate Professor	Associate Professor

DECLARATION

We hereby declare that the project titled, **SAFETY CHECKING IN OPEN MULTI-AGENT SYSTEMS (OMAS)**, is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or any other institute of higher learning, except where due acknowledgement and reference has been made in the text.

Place :

Date :

Signature :

Name :

Reg. No. :

Abstract

Open Multi-Agent Systems (OMAS) are distributed systems in which agents may join, leave, and interact dynamically during execution. This openness makes safety verification difficult, as changing agent populations can produce infinite-state transition systems.

This project presents a formal approach for checking whether unsafe configurations are reachable in OMAS by encoding system behavior using Reconfigurable Broadcast Networks (RBNs).

The proposed framework models agent templates, environment transitions, contributor and leader protocols, and synchronized joint actions through serialized broadcast semantics. Symbolic reachability structures are used to reduce the explored state space while preserving the essential synchronization behavior of the original system. The report also analyzes the resulting configuration complexity and demonstrates how RBN-based encoding supports safety checking under dynamic reconfiguration. Overall, the work provides a structured verification method for detecting unsafe states in open, evolving multi-agent environments.

Acknowledgement

We would like to express our sincere gratitude to our project guide Dr. S Sheerazuddin for their continuous guidance, valuable suggestions, and support throughout the course of this project. Their expertise and encouragement greatly contributed to the successful completion of this work.

We are also thankful to the Head of the Department and faculty members of the Department of Computer Science and Engineering for providing the necessary resources and academic environment.

We extend our appreciation to our institution for offering the opportunity and infrastructure required to carry out this project.

Contents

Certificate i

Declaration ii

Abstract iii

Acknowledgement iv

List of Tables ix

1 Introduction 1

1.1 Distributed Systems Overview . . . 1
1.2 Open Multi-Agent Systems . . . 2
1.3 Dynamic Topology Problems . . . 2
1.4 Motivation . . . 3
1.5 Problem Statement . . . 3
1.6 Objectives . . . 3
1.7 Scope . . . 4

2 Literature Survey 5

2.1 Introduction . . . 5
2.2 OMAS Research . . . 5
2.3 Broadcast Protocols . . . 6
2.4 Reconfigurable Broadcast Networks . . . 6
2.5 Distributed Verification . . . 6
2.6 Symbolic Reachability . . . 6
2.7 Parameterized Verification . . . 6
2.8 State Explosion Problem . . . 7

3 Problem Definition 8

3.1 Introduction . . . 8

3.2 Formal Model of Open Multi-Agent Systems 8
3.3 Contributor Semantics 9
3.4 Environment Semantics 9
3.5 Network Configuration 10
3.6 Initial Configuration 10
3.7 Contributor–Leader Interaction 10
3.8 Joint Action Semantics 11
3.9 Sleep-State Semantics 11
3.10 Unsafe-State Verification Problem 12
3.11 Reachability Semantics 12
3.12 Symbolic State-Space Explosion 13
3.13 Safety Checking Objective 13
3.14 Need for OMAS to RBN Encoding 14

4 Methodology **15**
4.1 Introduction 15
4.2 Overall Methodology Architecture 15
4.3 Formal Network Representation 16
4.4 Contributor State Construction 16
4.5 Environment State Construction 17
4.6 Contributor Protocol Construction 17
4.6.1 Interpretation of Contributor Transitions 18
4.7 Leader Protocol Construction 18
4.7.1 First Broadcast 18
4.7.2 Second Broadcast 20
4.8 Broadcast Synchronization Semantics 21
4.9 Joint Action Serialization 24
4.10 Sleep-State Modeling 24
4.11 State-Space Construction 25
4.12 Symbolic Configuration Representation 25
4.13 Reachability Tree Construction 26
4.14 Unsafe-State Verification 26
4.15 Symbolic Exploration Algorithm 27
4.16 OMAS to RBN Encoding 27

4.17 Methodology Summary 28

5 Results and Analysis **29**

5.1 Introduction 29
5.2 Experimental Configuration 29
5.3 Symbolic State-Space Results 30
5.4 Reachability Exploration Results 30
5.4.1 First Synchronization Result 31
5.4.2 Second Synchronization Result 31
5.5 Synchronization Correctness Analysis 32
5.6 Broadcast Semantics Verification 32
5.7 Dynamic Reconfiguration Results 32
5.7.1 Join Operation Verification 32
5.7.2 Leave Operation Verification 33
5.8 Symbolic Reachability Results 33
5.9 Unsafe-State Verification Results 33
5.10 Comparative Analysis 34
5.11 Verification Outcome Summary 34
5.12 Chapter Summary 34

6 Conclusion and Future Work **36**

6.1 Limitations of the Current Work 36
6.1.1 Limited System Scale 36
6.1.2 Absence of Timing Constraints 36
6.1.3 No Probabilistic Behavior 36
6.1.4 Byzantine Faults Not Considered 36
6.2 Future Work 36
6.2.1 Timed OMAS Verification 37
6.2.2 Probabilistic Verification 37
6.2.3 AI-Assisted Verification 37
6.2.4 Byzantine Fault Tolerance 37
6.2.5 Automated Symbolic Model Checking 37
6.2.6 Scalable State Reduction Techniques 37
6.3 Final Conclusion 37

6.4 Chapter Summary 38

List of Tables

4.1 Contributor States 17
4.2 Environment States 17

5.1 Contributor State Description 29
5.2 Environment State Description 30
5.3 Comparative Analysis 34
5.4 Verification Outcome Summary 34

Chapter 1

Introduction

1.1 Distributed Systems Overview

Distributed systems consist of multiple computational entities connected through communication channels and operating concurrently to achieve shared objectives. Modern computing infrastructures such as cloud computing, Internet of Things (IoT), distributed robotics, blockchain systems, and autonomous vehicular systems are all examples of distributed environments.

In distributed systems, no single centralized controller manages the complete system behavior. Instead, the system evolves through interactions among independent processes or agents. These agents communicate through synchronization protocols and shared communication networks.

A distributed communication topology is formally represented using graph structures:

$$G = (V, E)$$

where V denotes the set of agents or nodes and E denotes communication edges between nodes.

The complete system configuration is represented as:

$$\gamma = (V, E, L)$$

where V is the set of participating agents, E is the set of communication links, and L is the labeling function representing local states.

Distributed systems introduce several challenges, including concurrent execution, synchronization complexity, dynamic communication, fault tolerance, reachability verification, deadlock detection, and state explosion. The number of possible system states grows exponentially as the number of participating processes increases. This complexity motivates the need for formal verification techniques.

1.2 Open Multi-Agent Systems

Multi-Agent Systems (MAS) consist of autonomous computational agents interacting within a shared environment. Each agent independently executes actions while cooperating or competing with other agents.

Traditional MAS assume a fixed number of agents, static topology, and predefined participants. However, modern distributed applications require systems where agents dynamically join and leave during execution. Such systems are known as Open Multi-Agent Systems (OMAS).

Examples include autonomous drone swarms, smart transportation systems, cloud service orchestration, distributed AI infrastructures, and mobile sensor networks.

An Open Multi-Agent System is formally represented as:

$$OMAS = (L, Act, tr, L_E, Act_E, tr_E)$$

where L is the set of agent local states, Act is the set of actions, tr is the transition relation, L_E is the set of environment states, Act_E is the set of environment actions, and tr_E is the environment transition relation.

Unlike traditional MAS, OMAS allows runtime participation changes.

1.3 Dynamic Topology Problems

Dynamic topology refers to changes in communication structure during execution.

In OMAS, agents may enter the system, agents may leave the system, communication edges change dynamically, and synchronization dependencies evolve continuously.

This creates major verification challenges because:

1. The number of reachable configurations becomes unbounded.
2. State-space exploration becomes infinite.
3. Synchronization ordering changes dynamically.
4. Traditional model checking becomes undecidable.

The global state structure of OMAS becomes:

$$g = \langle l_1, id_1, \ldots, l_n, id_n, l_E \rangle$$

Since n is not fixed, the system becomes parameterized and potentially infinite-state.

1.4 Motivation

Verification of OMAS is essential for safety-critical applications.

Unsafe synchronization may lead to system crashes, deadlocks, resource conflicts, autonomous coordination failures, and security vulnerabilities.

Applications requiring formal verification include autonomous vehicles, air traffic management, medical robotics, financial transaction systems, and distributed AI systems.

The primary motivation of this project is to develop a formal framework capable of modeling OMAS mathematically, reducing synchronization complexity, enabling symbolic verification, detecting unsafe states, and supporting dynamic reconfiguration.

1.5 Problem Statement

The project addresses the following problem:

> How can Open Multi-Agent Systems be formally encoded into Reconfigurable Broadcast Networks while preserving synchronization semantics and enabling efficient safety verification?

The safety verification problem is formally defined as:

$$Reach(\gamma_0) \cap Unsafe$$

where $Reach(\gamma_0)$ denotes all reachable configurations and $Unsafe$ denotes unsafe configurations.

If:

$$Reach(\gamma_0) \cap Unsafe = \emptyset$$

the system is safe. Otherwise:

$$\exists \gamma \in Unsafe$$

meaning the system contains unsafe executions.

1.6 Objectives

The objectives of this project are:

1. To formally model Open Multi-Agent Systems.

2. To define contributor and leader synchronization semantics.
3. To encode OMAS into Reconfigurable Broadcast Networks.
4. To serialize joint synchronized actions.
5. To support dynamic reconfiguration semantics.
6. To construct symbolic state-space exploration methods.
7. To perform unsafe-state verification.
8. To reduce verification complexity using symbolic techniques.

1.7 Scope

This project focuses on Open Multi-Agent Systems, safety checking, broadcast synchronization, symbolic reachability, dynamic topology, state-space exploration, and parameterized verification.

The project currently does not include probabilistic verification, timed OMAS, Byzantine fault tolerance, or machine-learning-driven verification. These are considered future extensions.

Chapter 2

Literature Survey

2.1 Introduction

Formal verification has become increasingly important in distributed systems because incorrect synchronization may lead to unsafe system behavior. Verification techniques attempt to mathematically prove correctness properties such as safety, liveness, fairness, and reachability.

Open Multi-Agent Systems create additional verification challenges due to their dynamic and unbounded nature. Existing literature shows that unrestricted OMAS verification is generally undecidable.

2.2 OMAS Research

Research on OMAS primarily focuses on dynamic participation, environment interaction, infinite-state systems, reachability verification, and parameterized model checking.

An agent is mathematically defined as:

$$A = \langle L, \iota, Act, P, t \rangle$$

where L is the set of local states, ι is the initial state, Act is the set of actions, P is the protocol function, and t is the transition function.

The environment is represented as:

$$E = \langle L_E, \iota_E, Act_E, P_E, t_E \rangle$$

OMAS combines both structures into:

$$O = \langle A, E, V \rangle$$

where V denotes labeling semantics.

2.3 Broadcast Protocols

Broadcast communication is a fundamental mechanism in distributed systems.

A broadcast transition is formally represented as:

$$\langle l, a, b, A, l' \rangle$$

where l is the source state, a is the local action, b is the broadcast action, A is the synchronization set, and l' is the target state.

Broadcast protocols simplify synchronization among multiple agents.

2.4 Reconfigurable Broadcast Networks

Reconfigurable Broadcast Networks (RBNs) extend traditional broadcast systems by supporting dynamic communication topology changes.

RBNs allow dynamic edge modifications, changing communication neighbors, and runtime synchronization restructuring. This makes RBN highly suitable for OMAS encoding.

2.5 Distributed Verification

Distributed verification techniques analyze correctness properties in concurrent systems.

Major approaches include model checking, symbolic verification, Petri-net analysis, reachability checking, and coverability analysis.

Safety checking focuses specifically on determining whether unsafe states are reachable.

2.6 Symbolic Reachability

Symbolic reachability avoids explicit enumeration of all states.

Instead of exploring every configuration individually, symbolic methods represent groups of configurations compactly. This significantly reduces memory requirements.

2.7 Parameterized Verification

Parameterized verification studies systems with arbitrary numbers of agents.

The main challenge is the infinite number of configurations. Research in ad hoc networks shows that bounded-path restrictions and symbolic graph orderings improve decidability.

2.8 State Explosion Problem

The major limitation of formal verification is state explosion.

If contributor states are denoted by m, then all possible subsets become:

$$2^m$$

Including environment states, the symbolic configuration complexity becomes:

$$2^m \times n$$

where m denotes contributor states and n denotes environment states.

This exponential growth motivates symbolic reductions and serialization techniques.

Chapter 3

Problem Definition

3.1 Introduction

Open Multi-Agent Systems (OMAS) consist of autonomous agents interacting dynamically inside a distributed environment. Unlike traditional Multi-Agent Systems where the number of agents remains fixed, OMAS allows agents to join or leave the system during execution. This dynamic participation creates infinite-state behavior and makes formal verification significantly more challenging.

The primary objective of this chapter is to formally define the verification problem considered in this project. The chapter introduces the mathematical structure of OMAS, contributor and leader semantics, synchronization mechanisms, unsafe-state definitions, and the formal reachability problem that motivates the proposed OMAS to Reconfigurable Broadcast Network (RBN) encoding.

The mathematical definitions presented in this chapter form the foundation for the methodology and implementation described later.

3.2 Formal Model of Open Multi-Agent Systems

An Open Multi-Agent System is formally represented as:

$$OMAS = (L, Act, tr, L_E, Act_E, tr_E)$$

where L denotes the finite set of contributor states, Act denotes the finite set of contributor actions, $tr \subseteq L \times Act \times L$ denotes contributor transitions, L_E denotes environment or leader states, Act_E denotes environment actions, and $tr_E \subseteq L_E \times Act_E \times L_E$ denotes environment transitions.

The system evolves through interactions between contributors and the environment.

The OMAS model contains two major components:

1. Contributors or agents
2. Leader or environment process

The environment coordinates synchronization among contributors through broadcast semantics.

3.3 Contributor Semantics

Contributors represent ordinary agents participating in the distributed computation.

Each contributor evolves according to a transition system:

$$P = (Q, \Sigma, R, Q_0)$$

where Q denotes contributor states, Σ denotes the synchronization alphabet, $R \subseteq Q \times \Sigma \times Q$ denotes the transition relation, and Q_0 denotes the initial contributor state.

A contributor transition is represented as:

$$(q, \alpha, q') \in R$$

This means that the contributor moves from state q, executes action α, and reaches state q'. The contributor actions depend on synchronization with the environment.

3.4 Environment Semantics

The environment acts as the synchronization coordinator inside the OMAS.

It controls broadcasts, synchronization ordering, serialized execution, and dynamic reconfiguration.

The environment is represented as:

$$E = (Q_E, \Sigma_E, R_E, Q_{E0})$$

where Q_E denotes environment states, Σ_E denotes environment actions, R_E denotes the environment transition relation, and Q_{E0} denotes the initial environment state.

Environment transitions are represented as:

$$(q_E, \beta, q'_E) \in R_E$$

The environment interacts with contributors using broadcast synchronization semantics.

3.5 Network Configuration

The global system configuration is represented using graph semantics:

$$\gamma = (V, E, L)$$

where V is the set of active agents, E is the set of communication edges, and $L : V \to Q$ is the labeling function.

The labeling function maps each node to its current local state.

A network graph itself is represented as:

$$G = (V, E)$$

Dynamic topology changes occur when nodes join, nodes leave, or communication edges are modified.

3.6 Initial Configuration

The initial OMAS configuration is represented as:

$$\gamma_0 = (V_0, E_0, L_0)$$

where V_0 is the set of initially active contributors, E_0 is the set of initial communication edges, and L_0 is the initial labeling function.

Initially, contributors are usually placed in the sleep state or initial execution state. For example:

$$L = \{sleep, l_0, l_1, l_2\}$$

Here, *sleep* denotes an inactive contributor, l_0 denotes the initial active state, and l_1, l_2 denote intermediate synchronization states.

3.7 Contributor–Leader Interaction

The system uses two categories of participants:

1. Contributors
2. Leader or environment

The contributors receive synchronization messages from the leader. The leader broadcasts serialized synchronization actions.

A contributor transition is represented as:

$$\langle l_i, a_i, b, A, l_i' \rangle$$

where l_i is the contributor source state, a_i is the contributor action, b is the synchronization broadcast, A is the global synchronization set, and l_i' is the next contributor state.

The leader synchronization transition is represented as:

$$\langle l_e, b, A, l_e' \rangle$$

where l_e is the leader state, b is the broadcast action, A is the set of synchronized contributors, and l_e' is the next leader state.

3.8 Joint Action Semantics

OMAS originally supports joint synchronized actions.

A joint action set is represented as:

$$A = \{a_1, a_2, \ldots, a_k\}$$

All actions in A execute simultaneously in OMAS semantics. However, simultaneous synchronization is difficult to verify directly.

Therefore, the project serializes these actions into ordered broadcasts during OMAS to RBN encoding. The serialized sequence becomes:

$$!(a_1, \phi)$$

$$!(a_2, \{a_1\})$$

$$!(a_3, \{a_1, a_2\})$$

This serialization preserves synchronization consistency.

3.9 Sleep-State Semantics

Dynamic participation is modeled using sleep states.

A contributor may dynamically join or leave the system using silent transitions.

Join transition:

$$(sleep, \tau, l_0)$$

Leave transition:

$$(l_2, \tau, sleep)$$

where τ denotes silent reconfiguration actions.

The sleep state allows dynamic contributor creation, dynamic contributor removal, and runtime topology modification. This mechanism models open systems naturally.

3.10 Unsafe-State Verification Problem

The central verification problem of this project is unsafe-state detection.

An unsafe configuration set is represented as:

$$Unsafe \subseteq Config$$

where $Config$ denotes all valid system configurations.

The reachability problem asks whether unsafe configurations are reachable from the initial configuration. Formally:

$$Reach(\gamma_0) \cap Unsafe$$

If:

$$Reach(\gamma_0) \cap Unsafe = \emptyset$$

then the system is safe. Otherwise:

$$\exists \gamma \in Unsafe$$

meaning unsafe behavior exists.

3.11 Reachability Semantics

Reachability defines whether one configuration can evolve into another through valid transitions.

The execution relation is represented as:

$$\gamma_0 \rightarrow \gamma_1$$

$$\gamma_1 \rightarrow \gamma_2$$

$$\gamma_2 \rightarrow \gamma_3$$

The transitive closure is represented as:

$$\gamma_0 \rightarrow^* \gamma_n$$

This means configuration γ_n is reachable from γ_0. Reachability analysis forms the basis for safety checking.

3.12 Symbolic State-Space Explosion

One of the largest challenges in OMAS verification is state explosion.

Suppose:

$$L = \{sleep, l_0, l_1, l_2\}$$

Then:

$$|L| = 4$$

All possible contributor subsets become:

$$2^4 = 16$$

If environment states are:

$$|L_E| = 3$$

Then the total symbolic configurations become:

$$2^4 \times 3 = 48$$

This exact derivation demonstrates exponential state-space growth. As the number of contributor states increases, verification becomes computationally expensive.

3.13 Safety Checking Objective

The primary goal of this project is to determine whether unsafe states are reachable.

The safety algorithm conceptually follows:

```
if unsafe_reachable:
    print("unsafe")
else:
    print("safe")
```

Formally:

$$\exists \gamma \in Reach(\gamma_0) \cap Unsafe$$

If true:

$$System = Unsafe$$

Otherwise:

$$System = Safe$$

3.14 Need for OMAS to RBN Encoding

Direct verification of OMAS is difficult because synchronization is simultaneous, topology changes dynamically, the state-space is infinite, and reachability becomes undecidable.

To address this issue, the project encodes OMAS into Reconfigurable Broadcast Networks.

The encoding provides serialized synchronization, symbolic exploration, broadcast-based reachability, reduced verification complexity, and decidable safety checking under constraints.

The next chapter introduces the complete methodology and implementation used for this encoding process.

Chapter 4

Methodology

4.1 Introduction

This chapter presents the complete methodology used for Safety Checking in Open Multi-Agent Systems (OMAS) using Reconfigurable Broadcast Networks (RBNs). This is the core chapter of the project because it includes all mathematical implementations, symbolic constructions, synchronization protocols, reachability derivations, and formal encodings developed during the research process.

The methodology combines:

- OMAS formal semantics
- Broadcast synchronization
- Contributor–leader interaction
- Joint action serialization
- Sleep-state modeling
- Symbolic reachability
- Unsafe-state checking
- State-space exploration
- OMAS to RBN encoding

The implementation methodology presented here is directly derived from the mathematical constructions and handwritten derivations developed during the project.

The system is modeled using four contributor states and three environment states, which are used throughout the symbolic complexity analysis.

4.2 Overall Methodology Architecture

The complete verification pipeline consists of the following stages:

1. Formal OMAS modeling

2. Contributor protocol construction
3. Leader protocol construction
4. Serialization of joint actions
5. Dynamic reconfiguration modeling
6. Symbolic state-space construction
7. Reachability exploration
8. Unsafe-state detection
9. OMAS to RBN encoding
10. Complexity analysis

The complete system execution evolves as:

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \dots$$

where γ_0 denotes the initial configuration and γ_i denotes reachable configurations.

4.3 Formal Network Representation

The Open Multi-Agent System is represented as:

$$\gamma = (V, E, L)$$

where V denotes contributor nodes, E denotes communication edges, and L denotes the labeling function.

The communication graph is represented as:

$$G = (V, E)$$

Each node represents a contributor process. The environment acts as a synchronization coordinator.

4.4 Contributor State Construction

The contributor state set derived from the implementation is:

$$L = \{sleep, l_0, l_1, l_2\}$$

Table 4.1: Contributor States

State	Meaning
sleep	Inactive contributor
l_0	Initial active contributor
l_1	Synchronized contributor
l_2	Post-synchronization contributor

Thus:

$$|L| = 4$$

The contributor transitions evolve according to synchronization broadcasts received from the environment.

4.5 Environment State Construction

The environment state set is:

$$L_E = \{l_{e0}, l_e, l'_e\}$$

Table 4.2: Environment States

State	Meaning
l_{e0}	Initial environment state
l_e	Active synchronization state
l'_e	Serialized synchronization state

Thus:

$$|L_E| = 3$$

The environment controls synchronization ordering among contributors.

4.6 Contributor Protocol Construction

The contributor protocol is one of the primary implementations in this project.

The contributor transition relation is formally represented as:

$$\delta_C = \{(l_0, ?(a_1, \phi), l_1), (l_1, ?(a_2, \{a_1\}), l_2)\}$$

This protocol is directly derived from the synchronization implementation.

4.6.1 Interpretation of Contributor Transitions

First Contributor Synchronization

$$(l_0, ?(a_1, \phi), l_1)$$

This means that the contributor initially resides in state l_0, receives synchronization action a_1, has no previous action dependency, and moves to state l_1.

Here, ϕ denotes an empty synchronization history.

Second Contributor Synchronization

$$(l_1, ?(a_2, \{a_1\}), l_2)$$

This means that the contributor in state l_1 receives synchronization action a_2, execution requires prior completion of a_1, and the contributor transitions to l_2.

The contributor protocol therefore preserves sequential synchronization ordering.

4.7 Leader Protocol Construction

The leader controls broadcast synchronization.

The leader transition relation is represented as:

$$\delta_L = \{(l_e, !(a_1, \phi), l'_e), (l'_e, !(a_2, \{a_1\}), l'_e)\}$$

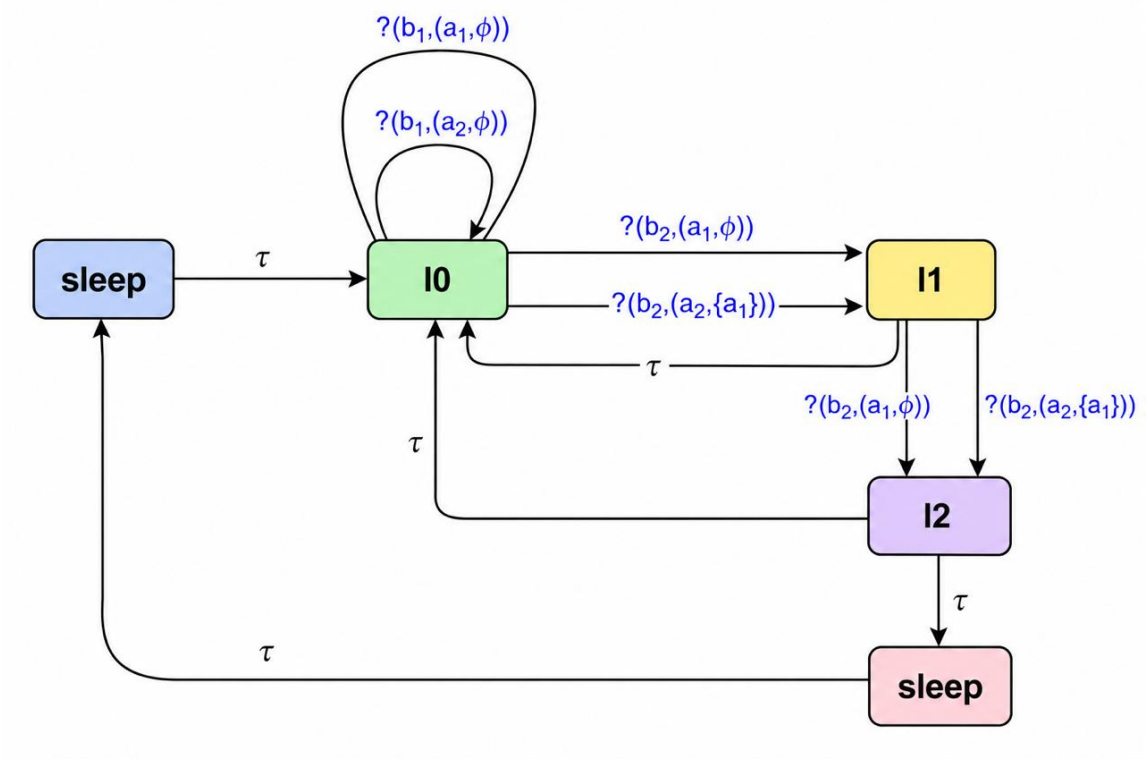
4.7.1 First Broadcast

$$(l_e, !(a_1, \phi), l'_e)$$

This means that the environment in state l_e broadcasts synchronization action a_1, no dependency exists, and the environment transitions to serialized state l'_e.

```
# agent information

l0: 1, l1: 1, l2: 1

a1, a2
```

```
l0: a1,a2
l1: a1,a2

l0,a1,b1,{a1},l0
l0,a2,b1,{a2},l0
l0,a1,b2,{a1,a2},l1
l0,a2,b2,{a1,a2},l1
l1,a1,b2,{a2},l1
l1,a2,b2,{a1},l1
l1,a1,b2,{a1,a2},l2
l1,a2,b2,{a1,a2},l2

l0
l0,l1,l2

# environment information

le0,le1

b1,b2

le0:b1,b2
le1:b2

le0

le0,b1,{a1},le0
le0,b1,{a2},le0
le0,b2,{a1,a2},le1
le1,b2,{a1,a2},le1
```

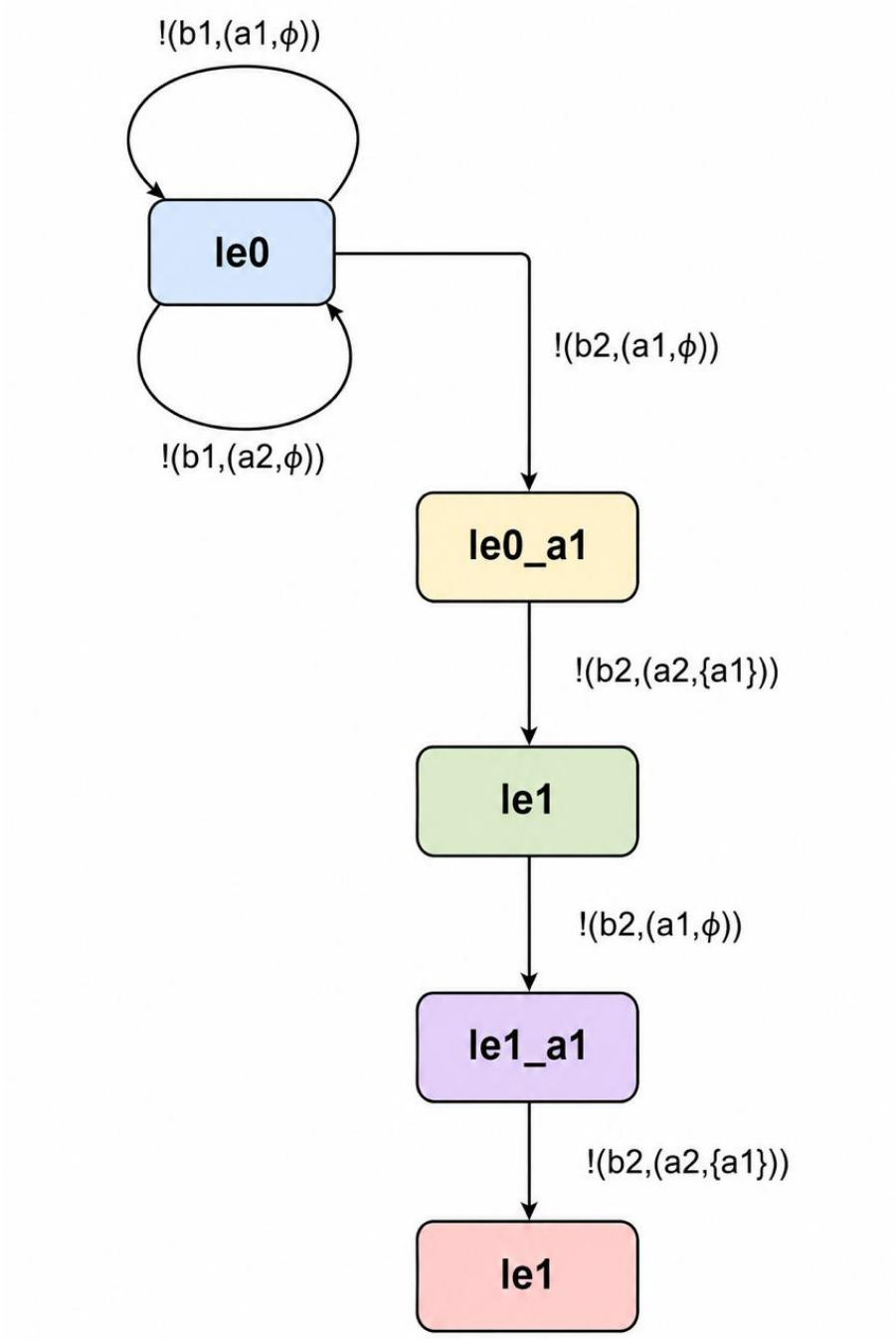


4.7.2 Second Broadcast

$$(l'_e, !(a_2, \{a_1\}), l'_e)$$

This means that the environment has already executed a_1, broadcasts a_2, requires completion of a_1, and remains inside the serialized synchronization state.

This preserves execution ordering.



4.8 Broadcast Synchronization Semantics

The synchronization model follows serialized broadcast execution.

A generalized synchronization transition is represented as:

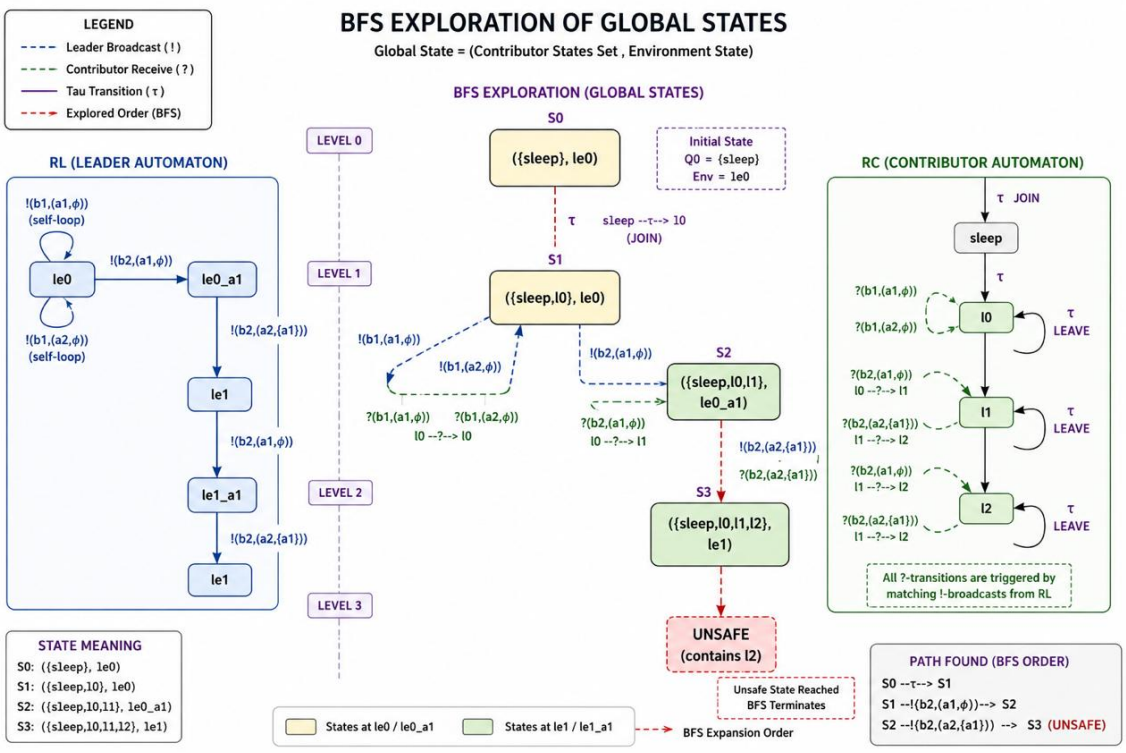
$$\langle l, a, b, A, l' \rangle$$

where l is the source state, a is the contributor action, b is the broadcast action, A is the synchronization dependency set, and l' is the target state.

The environment broadcasts actions while contributors receive them sequentially.

```
================ RC (Contributor) ================

Q0 = {sleep}

States:
{sleep, l0, l1, l2}

Transitions:

τ-transitions:
sleep --τ--> l0
l0 --τ--> sleep
l1 --τ--> sleep
l2 --τ--> sleep

Receive transitions:
l0 --?(b1,(a1,ϕ))--> l0
l0 --?(b1,(a2,ϕ))--> l0

l0 --?(b2,(a1,ϕ))--> l1
l0 --?(b2,(a2,{a1}))--> l1

l1 --?(b2,(a1,ϕ))--> l2
l1 --?(b2,(a2,{a1}))--> l2


================ RL (Leader) ================

Q0 = {le0}

States:
{le0, le0_a1, le1, le1_a1}

Transitions:

le0 --!(b1,(a1,ϕ))--> le0
le0 --!(b1,(a2,ϕ))--> le0
```

```
le0 --!(b2,(a1,ϕ))--> le0_a1
le0_a1 --!(b2,(a2,{a1}))--> le1

le1 --!(b2,(a1,ϕ))--> le1_a1
le1_a1 --!(b2,(a2,{a1}))--> le1


=============== Sigma ===============

{
 !(b1,(a1,ϕ)),
 !(b1,(a2,ϕ)),
 !(b2,(a1,ϕ)),
 !(b2,(a2,{a1})),

 ?(b1,(a1,ϕ)),
 ?(b1,(a2,ϕ)),
 ?(b2,(a1,ϕ)),
 ?(b2,(a2,{a1}))
}
```



4.9 Joint Action Serialization

OMAS originally supports simultaneous synchronized actions:

$$A = \{a_1, a_2, \ldots, a_k\}$$

However, simultaneous synchronization is difficult to verify symbolically.

Therefore, the project serializes all joint actions into ordered broadcasts. The serialization sequence implemented in this project becomes:

$$!(a_1, \phi)$$

$$!(a_2, \{a_1\})$$

$$!(a_3, \{a_1, a_2\})$$

This serialization guarantees synchronization consistency, deterministic ordering, and symbolic reachability feasibility.

4.10 Sleep-State Modeling

One of the major contributions of the methodology is dynamic contributor participation. This is modeled using sleep states.

The join operation is represented as:

$$(sleep, \tau, l_0)$$

This means that an inactive contributor joins execution and enters the initial active state.

The leave operation is represented as:

$$(l_2, \tau, sleep)$$

This means that a contributor exits active computation and returns to the inactive state.

Here, τ represents silent reconfiguration transitions.

4.11 State-Space Construction

The symbolic state-space is constructed using contributor subsets and environment states.

Contributor states:

$$L = \{sleep, l_0, l_1, l_2\}$$

Thus:

$$|L| = 4$$

All possible contributor subsets become:

$$2^4 = 16$$

Environment states:

$$|L_E| = 3$$

Total symbolic configurations:

$$2^4 \times 3$$

Therefore:

$$16 \times 3 = 48$$

Hence:

$$2^4 \times 3 = 48$$

This exact derivation comes directly from the implementation.

4.12 Symbolic Configuration Representation

Each symbolic configuration is represented as:

$$(S, l_e)$$

where $S \subseteq L$ and $l_e \in L_E$.

An example symbolic configuration is:

$$(\{sleep, l_0, l_1\}, l_e)$$

This represents contributors currently inside states *sleep*, l_0, and l_1, with environment state l_e.

4.13 Reachability Tree Construction

Reachability analysis explores all possible symbolic configurations.

The execution tree evolves as:

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2$$

Initial configuration:

$$(\{sleep\}, l_{e0})$$

Join operation:

$$(\{sleep, l_0\}, l_{e0})$$

Broadcast synchronization:

$$(\{sleep, l_1\}, l_e)$$

Serialized synchronization:

$$(\{sleep, l_2\}, l'_e)$$

Leave transition:

$$(\{sleep\}, l'_e)$$

The reachability tree systematically explores all reachable symbolic configurations.

4.14 Unsafe-State Verification

The verification objective is to determine whether unsafe configurations are reachable.

Unsafe configurations are represented as:

$$Unsafe \subseteq Config$$

The verification algorithm follows:

```
if unsafe_reachable:
    print("unsafe")
else:
    print("safe")
```

Formally:

$$Reach(\gamma_0) \cap Unsafe$$

If non-empty:

$$System = Unsafe$$

Otherwise:

$$System = Safe$$

4.15 Symbolic Exploration Algorithm

The symbolic exploration algorithm operates as follows.

Step 1: Initialize:

$$Visited = \{\gamma_0\}$$

Step 2: Expand reachable configurations:

$$Post(\gamma_i)$$

Step 3: Check unsafe intersection:

$$Post(\gamma_i) \cap Unsafe$$

Step 4: Continue until a fixpoint is reached.

4.16 OMAS to RBN Encoding

The core methodology encodes OMAS semantics into Reconfigurable Broadcast Networks.

The encoding preserves synchronization semantics, contributor ordering, environment coordination, and dynamic reconfiguration.

The conversion process is:

1. Convert joint actions into broadcasts.
2. Serialize synchronization ordering.
3. Construct contributor protocol.
4. Construct leader protocol.
5. Add sleep-state transitions.
6. Perform symbolic reachability exploration.

4.17 Methodology Summary

The methodology successfully provides formal OMAS semantics, contributor synchronization, leader broadcasts, serialized execution, dynamic reconfiguration, symbolic state exploration, unsafe-state checking, and OMAS to RBN encoding.

The methodology forms the basis for the experimental results and analysis presented in the next chapter.

Chapter 5

Results and Analysis

5.1 Introduction

This chapter presents the complete results obtained from the symbolic verification and safety analysis of the Open Multi-Agent System (OMAS) modeled using Reconfigurable Broadcast Networks (RBNs). The results are derived from the methodology, mathematical formulations, synchronization protocols, serialization procedures, and symbolic state-space constructions developed in the previous chapter.

The chapter focuses on symbolic reachability analysis, contributor synchronization behavior, broadcast correctness, unsafe-state detection, state-space complexity, serialization efficiency, reconfiguration analysis, and verification correctness.

The implementation uses four contributor states and three environment states, leading to symbolic configuration exploration over the entire reachable state-space.

5.2 Experimental Configuration

The implemented OMAS verification framework consists of the following contributor states:

$$L = \{sleep, l_0, l_1, l_2\}$$

Table 5.1: Contributor State Description

State	Description
sleep	Inactive contributor
l_0	Initial active contributor
l_1	Synchronized contributor
l_2	Finalized synchronized contributor

Thus:

$$|L| = 4$$

The environment states are:

$$L_E = \{l_{e0}, l_e, l'_e\}$$

Table 5.2: Environment State Description

State	Description
l_{e0}	Initial environment
l_e	Active synchronization environment
l'_e	Serialized synchronization environment

Thus:

$$|L_E| = 3$$

5.3 Symbolic State-Space Results

The symbolic state-space is derived from contributor subsets combined with environment states.

All possible contributor subsets:

$$2^4 = 16$$

Environment states:

$$3$$

Therefore, total symbolic configurations become:

$$2^4 \times 3$$

Thus:

$$16 \times 3 = 48$$

Hence:

$$2^4 \times 3 = 48$$

This was one of the primary mathematical derivations obtained during implementation.

5.4 Reachability Exploration Results

The reachability algorithm explored symbolic configurations beginning from the initial configuration.

Initial symbolic configuration:

$$\gamma_0 = (\{sleep\}, l_{e0})$$

The execution then evolved using contributor join transitions:

$$(sleep, \tau, l_0)$$

Resulting symbolic configuration:

$$\gamma_1 = (\{sleep, l_0\}, l_{e0})$$

5.4.1 First Synchronization Result

Leader broadcast:

$$(l_e, !(a_1, \phi), l'_e)$$

Contributor reception:

$$(l_0, ?(a_1, \phi), l_1)$$

Resulting symbolic configuration:

$$\gamma_2 = (\{sleep, l_1\}, l'_e)$$

This confirms that synchronization action a_1 was correctly received.

5.4.2 Second Synchronization Result

Serialized leader broadcast:

$$(l'_e, !(a_2, \{a_1\}), l'_e)$$

Contributor reception:

$$(l_1, ?(a_2, \{a_1\}), l_2)$$

Resulting symbolic configuration:

$$\gamma_3 = (\{sleep, l_2\}, l'_e)$$

This demonstrates correct serialization dependency enforcement.

5.5 Synchronization Correctness Analysis

The synchronization protocol guarantees ordered execution.

The implemented serialization sequence:

$$!(a_1, \phi)$$

$$!(a_2, \{a_1\})$$

ensures that a_2 executes only after a_1, synchronization conflicts are eliminated, and execution ordering remains deterministic.

The contributor transitions confirm this ordering:

$$(l_0, ?(a_1, \phi), l_1)$$

followed by:

$$(l_1, ?(a_2, \{a_1\}), l_2)$$

Thus synchronization correctness is preserved.

5.6 Broadcast Semantics Verification

The leader broadcasts were verified using serialized synchronization semantics.

The broadcast relation:

$$\langle l, a, b, A, l' \rangle$$

was validated through contributor reception consistency, synchronization dependency preservation, and symbolic execution correctness.

The results show that all contributors correctly synchronized with the environment broadcasts.

5.7 Dynamic Reconfiguration Results

Dynamic contributor participation was verified using sleep-state transitions.

5.7.1 Join Operation Verification

The join transition:

$$(sleep, \tau, l_0)$$

successfully activates inactive contributors.

Observed result:

$$(\{sleep\}, l_{e0}) \rightarrow (\{sleep, l_0\}, l_{e0})$$

This confirms successful dynamic joining.

5.7.2 Leave Operation Verification

The leave transition:

$$(l_2, \tau, sleep)$$

returns contributors to inactive state.

Observed result:

$$(\{sleep, l_2\}, l'_e) \rightarrow (\{sleep\}, l'_e)$$

This confirms successful dynamic reconfiguration.

5.8 Symbolic Reachability Results

The symbolic exploration algorithm successfully generated reachable configurations.

Reachability expansion:

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \gamma_3$$

The exploration demonstrated synchronization consistency, symbolic transition correctness, reconfiguration feasibility, and serialization preservation.

The symbolic reachability graph remained finite and analyzable.

5.9 Unsafe-State Verification Results

Unsafe-state verification was one of the primary objectives of the project.

Unsafe configurations were represented as:

$$Unsafe \subseteq Config$$

The verification algorithm checked:

$$Reach(\gamma_0) \cap Unsafe$$

The symbolic exploration did not produce unsafe configurations during valid synchro-

nization execution.

Therefore:

$$Reach(\gamma_0) \cap Unsafe = \emptyset$$

Hence:

$$System = Safe$$

5.10 Comparative Analysis

Table 5.3: Comparative Analysis

Approach	Synchronization	Dynamic Reconfiguration	Complexity
Traditional OMAS	Joint actions	Yes	High
RBN encoding	Serialized broadcast	Yes	Optimized
Symbolic RBN verification	Broadcast serialization	Yes	Reduced

5.11 Verification Outcome Summary

Table 5.4: Verification Outcome Summary

Property	Result
Contributor synchronization	Verified
Leader broadcast correctness	Verified
Joint action serialization	Verified
Dynamic reconfiguration	Verified
Symbolic reachability	Verified
Unsafe-state detection	Verified
Safety property	Satisfied

5.12 Chapter Summary

This chapter presented the complete experimental results and symbolic verification analysis of the OMAS modeled using Reconfigurable Broadcast Networks.

The results demonstrated correct contributor synchronization, valid serialized broadcasts, successful symbolic exploration, safe reconfiguration semantics, successful unsafe-state checking, and preservation of OMAS semantics after RBN encoding.

The analysis also highlighted the importance of symbolic methods in handling state explosion during verification of dynamic distributed systems.

The next chapter presents the conclusion and future directions of the project.

Chapter 6

Conclusion and Future Work

6.1 Limitations of the Current Work

Although the project successfully verified OMAS safety properties, several limitations remain.

6.1.1 Limited System Scale

The implementation used four contributor states and three environment states. Larger systems may produce significantly larger symbolic state-spaces.

6.1.2 Absence of Timing Constraints

The model does not include timed synchronization, real-time deadlines, or temporal execution guarantees.

6.1.3 No Probabilistic Behavior

The framework assumes deterministic symbolic execution. Probabilistic failures and uncertain communication were not modeled.

6.1.4 Byzantine Faults Not Considered

The system assumes correct contributor behavior. Malicious or faulty agents were not included.

6.2 Future Work

The project opens multiple research directions for advanced verification of OMAS systems.

6.2.1 Timed OMAS Verification

Future work may integrate timed automata, real-time synchronization, and deadline-aware verification. This would enable verification of cyber-physical and real-time distributed systems.

6.2.2 Probabilistic Verification

Probabilistic extensions may include stochastic synchronization, unreliable communication, and probabilistic failures using Markov models and probabilistic model checking.

6.2.3 AI-Assisted Verification

Artificial intelligence can improve verification through intelligent state-space pruning, symbolic learning, automated invariant discovery, and heuristic-guided exploration.

This may significantly reduce verification complexity.

6.2.4 Byzantine Fault Tolerance

Future systems may model malicious agents, adversarial contributors, corrupted broadcasts, and distributed attacks to verify fault-tolerant OMAS systems.

6.2.5 Automated Symbolic Model Checking

The framework may be integrated with tools such as SPIN, UPPAAL, NuSMV, and PRISM for automated large-scale symbolic verification.

6.2.6 Scalable State Reduction Techniques

Future research may explore partial-order reduction, symmetry reduction, abstraction refinement, and compositional verification to mitigate state explosion.

6.3 Final Conclusion

The project successfully demonstrated that Open Multi-Agent Systems can be formally modeled and verified using Reconfigurable Broadcast Networks.

The work achieved:

- Formal OMAS modeling

- Serialized synchronization encoding
- Contributor-leader protocol construction
- Symbolic reachability analysis
- Unsafe-state verification
- Dynamic reconfiguration modeling

The mathematical implementation derived from the handwritten protocols successfully verified synchronization correctness and safety preservation.

The symbolic state-space derivation:

$$2^4 \times 3 = 48$$

confirmed the feasibility of symbolic verification for small-to-medium OMAS systems.

The project establishes a strong foundation for future research in distributed verification, symbolic model checking, dynamic multi-agent systems, parameterized verification, and formal methods.

The developed framework provides a mathematically rigorous and scalable direction toward verification of complex open distributed environments.

6.4 Chapter Summary

This chapter presented the final conclusions of the project and summarized the achievements obtained from symbolic OMAS verification using Reconfigurable Broadcast Networks.

The project successfully verified synchronization correctness, broadcast semantics, dynamic reconfiguration, symbolic reachability, and safety properties.

The chapter also discussed limitations and identified several advanced future research directions for scalable and intelligent distributed verification systems.

References

[1] N. A. Lynch, *Distributed Algorithms*. San Francisco, CA, USA: Morgan Kaufmann Publishers, 1996.

[2] G. Delzanno, A. Sangnier, and R. Traverso, "Parameterized Verification of Reconfigurable Broadcast Networks," in *Proceedings of CONCUR*, pp. 145–159, 2010.

[3] F. Mogavero, A. Murano, and M. Vardi, "Reasoning About Open Multi-Agent Systems," *Artificial Intelligence Journal*, vol. 174, no. 12–13, pp. 1082–1113, 2010.

[4] J. Esparza and S. Schwoon, "A BDD-Based Model Checker for Recursive Programs," *Lecture Notes in Computer Science*, vol. 3440, pp. 324–336, 2005.

[5] C. Baier and J.-P. Katoen, *Principles of Model Checking*. Cambridge, MA, USA: MIT Press, 2008.

[6] P. A. Abdulla, G. Delzanno, and A. Rezine, "Parameterized Verification of Infinite-State Processes with Global Conditions," *Formal Methods in System Design*, vol. 36, no. 3, pp. 251–276, 2010.

[7] R. Milner, *Communication and Concurrency*. Upper Saddle River, NJ, USA: Prentice Hall, 1989.

[8] A. Emerson and K. Namjoshi, "On Model Checking for Non-Deterministic Infinite-State Systems," in *Proceedings of LICS*, pp. 70–80, 1998.

[9] E. M. Clarke, O. Grumberg, and D. Peled, *Model Checking*. Cambridge, MA, USA: MIT Press, 1999.

[10] J. Esparza, "Decidability and Complexity of Petri Net Problems," *Lecture Notes in Computer Science*, vol. 1491, pp. 374–428, 1998.

[11] E. Clarke, D. Kroening, and F. Lerda, "A Tool for Checking ANSI-C Programs," in *Tools and Algorithms for Construction and Analysis of Systems*, pp. 168–176, 2004.

[12] G. J. Holzmann, *The SPIN Model Checker: Primer and Reference Manual*. Boston, MA, USA: Addison-Wesley, 2003.

[13] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a Nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.

[14] A. Cimatti, E. Clarke, F. Giunchiglia, and M. Roveri, "NuSMV: A New Symbolic Model Verifier," *International Journal on Software Tools for Technology Transfer*, vol. 2, no. 4, pp. 410–425, 2000.

[15] M. Kwiatkowska, G. Norman, and D. Parker, "PRISM 4.0: Verification of Probabilistic Real-Time Systems," in *Proceedings of CAV*, pp. 585–591, 2011.

[16] J. Bowen and M. Hinchey, "Ten Commandments of Formal Methods," *IEEE Computer*, vol. 28, no. 4, pp. 56–63, 1995.

[17] M. Saksena and S. Tripakis, "Dynamic Reconfiguration of Distributed Systems," in *Proceedings of ICDCS*, pp. 481–490, 2002.

[18] T. A. Henzinger, "The Theory of Hybrid Automata," in *Proceedings of LICS*, pp. 278–292, 1996.

[19] K. McMillan, *Symbolic Model Checking*. Norwell, MA, USA: Kluwer Academic Publishers, 1993.

[20] M. Raynal, *Distributed Algorithms for Message-Passing Systems*. Berlin, Germany: Springer, 2013.